

Key-Value Store v0

Nodo singolo, interfaccia e contratto

Architetture dei Sistemi Distribuiti

Lezione 09

Perche' partire da qui

Il key-value store e' un caso di studio ideale:

- modello dati minimale;
- comandi intuitivi;
- protocollo facile da osservare manualmente;
- stessa API riutilizzabile nelle lezioni successive;
- ottimo terreno per discutere promesse e limiti.

L'obiettivo di oggi non e' la complessita' implementativa. L'obiettivo e' fissare bene il significato di cio' che il server dice al client.

Tre livelli da non confondere

Interfaccia

Quali richieste il client puo' inviare.

Contratto

Quali garanzie semantiche riceve quando il server risponde.

Implementazione

Quale codice e quali strutture dati realizzano quel contratto.

Gran parte del corso consiste nel mostrare che questi tre livelli non coincidono automaticamente.

Domanda guida

Che cosa significa davvero che una SET $k \rightarrow v$ e' andata a buon fine?

Oggi la risposta e' molto debole:

- un processo e' vivo;
- ha aggiornato il proprio stato in RAM;
- ha spedito OK sulla connessione.

Domani la stessa domanda coinvolgera' disco, repliche, quorum, migrazione e versioni.

Che cos'è il v0

La versione iniziale è volutamente povera:

- un solo nodo;
- stato solo in memoria;
- protocollo testuale su TCP;
- nessuna persistenza;
- nessuna replica;
- nessuna tolleranza ai guasti.

Proprio per questo è utile: rende più visibile il confine tra ciò che il sistema fa e ciò che non promette affatto.

Scelte del v0:

- chiavi: stringhe senza spazi;
- valori: stringhe arbitrarie;
- trasporto: TCP;
- codifica: UTF-8;
- una richiesta per riga;
- una risposta per riga.

Semplice non significa neutro: ogni scelta restringe già' il tipo di client e di semantica che possiamo sostenere.

```
PING
OK PONG

SET corso asd
OK

GET corso
OK asd
```

Caratteristiche importanti:

- protocollo human-friendly;
- osservabile con telnet o nc;
- ottimo per discutere significato delle risposte.

Il servizio espone:

- PING
- SET key value
- GET key
- DELETE key
- EXISTS key
- KEYS
- QUIT

Sembra una lista ovvia. In realta' ciascun comando apre una discussione su semantica, errori e casi limite.

Alcune scelte apparentemente banali:

- chiave assente: NOT_FOUND o ERR?
- valore vuoto: distinto da chiave assente oppure no?
- DELETE su chiave assente: errore o esito applicativo?
- KEYS: parte del contratto principale o solo comando diagnostico?
- OK di SET: successo locale o qualcosa di piu' ?

Ogni risposta qui anticipa problemi molto piu' costosi nelle versioni distribuite.

```
GET corso
```

```
NOT_FOUND
```

```
SET corso
```

```
ERR usage: SET <key> <value>
```

```
DELETE corso
```

```
NOT_FOUND
```

Notare la differenza tra:

- errore di protocollo;
- esito applicativo legittimo;
- successo locale.

Cosa Promette Davvero OK

Nel v0, OK di una scrittura promette solo:

- update applicato al dizionario del processo;
- ordine coerente rispetto alla singola connessione;
- visibilita' immediata sullo stesso nodo finche' il processo resta vivo.

Non promette:

- durabilita';
- replica;
- sopravvivenza al crash;
- osservazione uniforme da parte di piu' nodi.

Per SET possiamo immaginare:

- Idle -> Parse -> Validate -> ApplyInMemory -> ReplyOK

Per GET:

- Idle -> Parse -> Lookup -> ReplyValue

L'automa e' povero, ma e' proprio questo che vogliamo: una base su cui inserire poi concorrenza, disco, replica e migrazione.

Perche' KEYS E' Gia' Interessante

Oggi KEYS sembra innocuo:

- stato locale;
- costo basso;
- semantica ovvia.

Domani diventera' ambiguo:

- su quale nodo?
- su quali repliche?
- su quali shard?
- con quale grado di completezza e consistenza?

Che Cosa Non Stiamo Ancora Affrontando

Restano completamente aperti:

- accessi concorrenti;
- crash recovery;
- commit distribuito;
- stale reads;
- failover;
- partizionamento del dataset.

Il valore del v0 sta proprio qui: isolare il primo contratto prima di stressarlo.

Suggerisco di far osservare almeno:

- ① chiave assente contro valore vuoto;
- ② richieste malformate;
- ③ ordine delle operazioni sulla stessa connessione;
- ④ uso di KEYS come operazione apparentemente semplice;
- ⑤ significato preciso di OK su SET.

Come Si Evolvera' Lo Stesso Contratto

Nelle prossime tappe la stessa interfaccia verra' messa sotto pressione da:

- multithreading e interleaving;
- persistenza locale;
- replica primaria-secondaria;
- failover;
- quorum;
- sharding e rebalancing.

La sintassi cambiera' poco. Il contenuto semantico delle risposte cambiera' molto.

La prima lezione sul KV store non parla davvero di dizionari Python. Parla di disciplina progettuale:

- definire l'interfaccia;
- rendere esplicito il contratto;
- distinguere cio' che il sistema fa da cio' che promette.

Se oggi non sapete spiegare bene OK, domani non saprete spiegare bene commit.